

```

1 def is_safe(board, row, col):
2     for i in range(row):
3         if board[i] == col or \
4             board[i] - i == col - row or \
5             board[i] + i == col + row:
6             return False
7     return True
8 def solve_n_queens_util(board, row):
9     if row == len(board):
10        return board
11    for col in range(len(board)):
12        if is_safe(board, row, col):
13            board[row] = col
14            solution = solve_n_queens_util(board, row + 1)
15            if solution:
16                return solution
17            board[row] = -1
18    return None
19 def solve_n_queens(n):
20     board = [-1] * n
21     return solve_n_queens_util(board, 0)
22 def print_solution(board):
23     n = len(board)
24     for row in range(n):
25         line = ['Q' if col == board[row] else '.' for col in range(n)]
26         print(' '.join(line))
27     print()
28 n = 8
29 solution = solve_n_queens(n)
30 if solution:
31     print_solution(solution)
32 else:
33     print("No solution found.")
34

```

Output

```

Q . . . . .
. . . Q . .
. . . . . Q
. . . . Q .
. Q . . . .
. . . . Q .
. Q . . . .
. . . Q . .

```

=== Code Execution Successful ===

main.py



Share

Run

Output

```
1 warehouse_graph = {  
2     'A': ['B', 'C'],  
3     'B': ['D', 'E'],  
4     'C': ['F'],  
5     'D': [],  
6     'E': ['F'],  
7     'F': []  
8 }  
9 def dfs(graph, start, goal, visited=None, path=None):  
10     if visited is None:  
11         visited = set()  
12     if path is None:  
13         path = []  
14     visited.add(start)  
15     path.append(start)  
16     if start == goal:  
17         return path  
18     for neighbor in graph[start]:  
19         if neighbor not in visited:  
20             result = dfs(graph, neighbor, goal, visited, path[:])  
21             if result:  
22                 return result  
23     return None  
24 start_node = 'A'  
25 goal_node = 'F'  
26 path_found = dfs(warehouse_graph, start_node, goal_node)  
27 print(f"DFS Path from {start_node} to {goal_node}: {path_found}")
```

DFS Path from A to F: ['A', 'B', 'E', 'F']

=== Code Execution Successful ===



main.py

```
1 PLAYER_X = 1
2 PLAYER_O = -1
3 EMPTY = 0
4 def evaluate(board):
5     for row in range(3):
6         if board[row][0] == board[row][1] == board[row][2] != EMPTY:
7             return board[row][0]
8     for col in range(3):
9         if board[0][col] == board[1][col] == board[2][col] != EMPTY:
10            return board[0][col]
11    if board[0][0] == board[1][1] == board[2][2] != EMPTY:
12        return board[0][0]
13    if board[0][2] == board[1][1] == board[2][0] != EMPTY:
14        return board[0][2]
15    return 0
16 def isMovesLeft(board):
17     for row in range(3):
18         for col in range(3):
19             if board[row][col] == EMPTY:
20                 return True
21     return False
22 def minimax(board, isMax):
23     score = evaluate(board)
24     if score == PLAYER_X: return score
25     if score == PLAYER_O: return score
26     if not isMovesLeft(board): return 0
```

Output

Current Board:

```
X O X
O X .
. O X
```

Best Move: (1, 2)

Board after best move:

```
X O X
O X X
. O X
```

=== Code Execution Successful ===

Upcoming
Earnings

Search

```

25  if score == PLAYER_O: return score
26  if not isMovesLeft(board): return 0
27  if isMax:
28      best = -float('inf')
29      for row in range(3):
30          for col in range(3):
31              if board[row][col] == EMPTY:
32                  board[row][col] = PLAYER_X
33                  best = max(best, minimax(board, not isMax))
34                  board[row][col] = EMPTY
35      return best
36  else:
37      best = float('inf')
38      for row in range(3):
39          for col in range(3):
40              if board[row][col] == EMPTY:
41                  board[row][col] = PLAYER_O
42                  best = min(best, minimax(board, not isMax))
43                  board[row][col] = EMPTY
44      return best
45  def findBestMove(board):
46      bestVal = -float('inf')
47      bestMove = (-1, -1)
48      for row in range(3):
49          for col in range(3):
50              if board[row][col] == EMPTY:

```

Run

Output

Current Board:

X O X

O X .

. O X

Best Move: (1, 2)

Board after best move:

X O X

O X X

. O X

=== Code Execution Successful ===

```

48- for row in range(3):
49-     for col in range(3):
50-         if board[row][col] == EMPTY:
51-             board[row][col] = PLAYER_X
52-             moveVal = minimax(board, False)
53-             board[row][col] = EMPTY
54-             if moveVal > bestVal:
55-                 bestMove = (row, col)
56-                 bestVal = moveVal
57-     return bestMove
58- def printBoard(board):
59-     for row in board:
60-         print(" ".join(["X" if x == PLAYER_X else "O" if x
61- ==PLAYER_O else "." for x in row]))
62- board = [
63-     [PLAYER_X, PLAYER_O, PLAYER_X],
64-     [PLAYER_O, PLAYER_X, EMPTY],
65-     [EMPTY, PLAYER_O, PLAYER_X]
66- ]
67- print("Current Board:")
68- printBoard(board)
69- move = findBestMove(board)
70- print(f"Best Move: {move}")
71- board[move[0]][move[1]] = PLAYER_X
72- print("\nBoard after best move:")
73- printBoard(board)

```

Output

Current Board:

X O X

O X .

. O X

Best Move: (1, 2)

Board after best move:

X O X

O X X

. O X

=== Code Execution Successful ===

main.py

```
1 import heapq
2 class Node:
3     def __init__(self, position, parent=None, g=0, h=0):
4         self.position = position
5         self.parent = parent
6         self.g = g
7         self.h = h
8         self.f = g
9     def __lt__(self, other):
10         return self.f < other.f
11 def heuristic(a, b):
12     return abs(a[0] - b[0]) + abs(a[1] - b[1])
13 def a_star(grid, start, goal):
14     rows, cols = len(grid), len(grid[0])
15     open_list = []
16     heapq.heappush(open_list, Node(start, None, 0, heuristic(start,
17         goal)))
18     closed_set = set()
19     while open_list:
20         current_node = heapq.heappop(open_list)
21         if current_node.position == goal:
22             path = []
23             while current_node:
24                 path.append(current_node.position)
25                 current_node = current_node.parent
26             return path[::-1]
```

Output

Optimal Path: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (3, 4), (4, 4)]

=== Code Execution Successful ===

Clear

31°C
Mostly cloudy

Search

ENG
IN

```

25     return path[::-1]
26
27     closed_set.add(current_node.position)
28     for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
29         new_pos = (current_node.position[0] + dr, current_node
30                     .position[1] + dc)
31         if (0 <= new_pos[0] < rows and 0 <= new_pos[1] < cols
32             and
33             grid[new_pos[0]][new_pos[1]] == 0 and new_pos not in
34                 closed_set):
35             new_node = Node(new_pos, current_node, current_node
36                             .g + 1, heuristic(new_pos,
37                                     goal))
38             heapq.heappush(open_list, new_node)
39
40     return None
41
42 warehouse_grid = [
43     [0, 0, 0, 0, 1],
44     [1, 1, 0, 1, 0],
45     [0, 0, 0, 0, 0],
46     [0, 1, 1, 1, 0],
47     [0, 0, 0, 0, 0]
48 ]
49
50 start_position = (0, 0)
51 goal_position = (4, 4)
52 path = a_star(warehouse_grid, start_position, goal_position)
53 print("Optimal Path:", path)

```

Output

Optimal Path: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (3, 4), (4, 4)]

=== Code Execution Successful ===

Clear

31°C
Mostly cloudy

Search

ENG
IN

main.py

```
1 def minimum(X, Y):
2     if X <= Y:
3         return X
4     else:
5         return Y
6
7 def maximum(X, Y):
8     if X >= Y:
9         return X
10    else:
11        return Y
12
13 num1 = float(input("Enter the first number: "))
14 num2 = float(input("Enter the second number: "))
15
16 min_value = minimum(num1, num2)
17 max_value = maximum(num1, num2)
18
19 print(f"Min: {min_value}")
20 print(f"Max: {max_value}")
21
```

```
Enter the first number: 20
Enter the second number: 50
Min: 20.0
Max: 50.0
```

```
=== Code Execution Successful ===
```




main.py



Share

Run

Output

```
1 import re
2 def unify(x, y, theta={}):
3     if theta is None:
4         return None
5     elif x == y:
6         return theta
7     elif isinstance(x, str) and x.islower():
8         return unify_var(x, y, theta)
9     elif isinstance(y, str) and y.islower():
10        return unify_var(y, x, theta)
11    elif isinstance(x, list) and isinstance(y, list) and len(x) == len(y):
12        return unify(x[1:], y[1:], unify(x[0], y[0], theta))
13    else:
14        return None
15 def unify_var(var, x, theta):
16     if var in theta:
17         return unify(theta[var], x, theta)
18     elif x in theta:
19         return unify(var, theta[x], theta)
20     else:
21         theta[var] = x
22         return theta
23 def resolution(kb, query):
24     for clause in kb:
25         theta = unify(clause[0], query, {})
26         if theta is not None:
27             new_kb = clause[1:]
28             if not new_kb:
29                 return True
30             else:
31                 return resolution(kb, new_kb[0])
32     return False
33 knowledge_base = [
34     ["Human", "John"], ["Mortal", "John"]],
35 ]
36 fact = ["Human", "John"]
37 query = ["Mortal", "John"]
38 if resolution(knowledge_base, query):
39     print("Query is resolved: John is Mortal")
40 else:
41     print("Query could not be resolved")
```

Query could not be resolved

=== Code Execution Successful ===

main.py



Share

Run

```
1 # Knowledge Base (Rules in IF-THEN format)
2 knowledge_base = {
3     "flu": [{"cough", "fever"}],
4     "fever": [{"sore_throat"}],
5 }
6 facts = {"sore_throat", "cough"}
7 def backward_chaining(goal):
8     if goal in facts:
9         return True
10    if goal in knowledge_base:
11        for conditions in knowledge_base[goal]:
12            if all(backward_chaining(cond) for cond in conditions):
13                return True
14    return False
15 query = "flu"
16 if backward_chaining(query):
17     print(f"The patient is diagnosed with {query}.")
18 else:
19     print(f"The patient does NOT have {query}.")
```

Output

Clear

The patient is diagnosed with flu.

=== Code Execution Successful ===

Google Ads

Drive sales,
be found, &
stand out

G-Your Business



ENG

100%



100%

28-05-2024



main.py



Share

Run

Output

Clear

```
1 knowledge_base = [  
2     ([ "cough", "fever"], "flu"),  
3     ([ "sore_throat", "runny_nose"], "cold"),  
4     ([ "sore_throat"], "fever")  
5 ]  
6 facts = { "cough", "sore_throat" }  
7 def forward_chaining():  
8     inferred = True  
9     while inferred:  
10         inferred = False  
11         for conditions, conclusion in knowledge_base:  
12             if all(condition in facts for condition in conditions) and  
13                 conclusion not in facts:  
14                 facts.add(conclusion)  
15                 inferred = True  
16 forward_chaining()  
17 if "flu" in facts:  
18     print("The patient is diagnosed with flu.")  
19 elif "cold" in facts:  
20     print("The patient is diagnosed with cold.")  
21 else:  
22     print("No conclusive diagnosis could be made.")
```

The patient is diagnosed with flu.

== Code Execution Successful ==



main.py



Share

Run

Output

Clear

```
1 class Blocksworld:
2     def __init__(self):
3         self.state = {
4             "A": "B",
5             "B": "table",
6             "C": "table"
7         }
8         self.goal = {
9             "A": "B",
10            "B": "C",
11            "C": "table"
12        }
13    def is_goal_state(self):
14        return self.state == self.goal
15    def move(self, block, destination):
16        if block in self.state and self.state[block] != destination:
17            print(f"Moving {block} from {self.state[block]} to {destination}")
18            self.state[block] = destination
19    def plan_moves(self):
20        print("\nInitial State:", self.state)
21        while not self.is_goal_state():
22            for block, target in self.goal.items():
23                if self.state[block] != target:
24                    self.move(block, target)
25        print("\nFinal Goal State Reached:", self.state)
26 bw = Blocksworld()
27 bw.plan_moves()
```

```
Initial State: {'A': 'B', 'C': 'table'}
Moving B from table to C
Moving A from B to B
Moving C from table to table
Final Goal State Reached: {'A': 'B', 'B': 'C', 'C': 'table'}
```

